

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Ing. Edgar Raul Molina Rey

Análisis de Sistemas



SISTEMA AUTOMATIZADO DE GESTIÓN DE FARMACIA

Proyecto Final – Análisis de Sistemas I

Anthony Jacob Barrios Pérez

0902-20-9735

Cobán, Alta Verapaz, 30 de mayo de 2026.

INDICE

INTRODUCCIÓN	4
PLANTEAMIENTO DEL PROBLEMA	5
OBJETIVOS	6
1. ARQUITECTURA DEL SISTEMA	7
<i>Imagen 1. Diagrama UML de Despliegue de la Arquitectura Local del Sistema</i>	7
2. TECNOLOGIAS USADAS	8
3. DESCRIPCIÓN DE LOS WORKFLOWS	9
3.1. WORKFLOW 1 – INGESTA DE INVENTARIO	9
<i>Imagen 2. Workflow de ingesta de inventario en n8n</i>	10
3.2. WORKFLOW 2 – PROCESAMIENTO DE REGLAS Y ALERTAS	11
<i>Imagen 3. Workflow de procesamiento de reglas y generación de alertas en n8n</i>	12
3.3. WORKFLOW 3 – GENERACIÓN DE REPORTES	13
<i>Imagen 4. Workflow de generación de reportes en n8n</i>	14
3.4. WORKFLOW 4 - MANEJO DE ERRORES	15
<i>Imagen 5. Workflow de manejo de errores en n8n</i>	16
4. BASE DE DATOS	17
4.1. DIAGRAMA RELACIONAL DE LA BASE DE DATOS	17
<i>Imagen 6. Diagrama relacional de la base de datos</i>	18
4.2. ESTRUCTURA DE LA TABLA PRODUCTOS	18
<i>Imagen 7. Estructura de la tabla productos</i>	18
4.3. ESTRUCTURA DE LA TABLA ALERTAS	19
<i>Imagen 8. Estructura de la tabla alertas</i>	19
4.4. INTEGRACIÓN DE POSTGRESQL CON N8N	19
<i>Imagen 9. Nodo PostgreSQL utilizado en n8n</i>	20
4.5. DATOS DE PRUEBA UTILIZADOS	20
<i>Imagen 10. Datos de prueba utilizados por el sistema</i>	20
5. CONSIDERACIONES DE SEGURIDAD	21
6. EVIDENCIAS DE EJECUCIÓN Y RESULTADOS DEL SISTEMA	22
6.1. PANEL PRINCIPAL DEL SISTEMA	22
<i>Imagen 11. Panel principal de pruebas WF1-WF4</i>	22
6.2. PRUEBAS DEL WORKFLOW WF1	23

<i>Imagen 12. Registro exitoso de ingreso de inventario (WF1)</i>	23
<i>Imagen 13. Configuración del proyecto en Supabase</i>	23
<i>Imagen 14. Ejecución exitosa del workflow WF1 – Ingesta de Inventario</i>	24
<i>Imagen 15. Tabla Productos en Supabase</i>	24
<i>Imagen 16. Tabla Movimientos en Supabase</i>	25
<i>Imagen 17. Tabla Audit Log</i>	25
6.3. PRUEBAS DEL WORKFLOW WF2	26
<i>Imagen 18. Ejecución exitosa del workflow WF2</i>	26
<i>Imagen 19. Alertas generadas automáticamente</i>	26
6.4. PRUEBAS DEL WORKFLOW WF3	27
<i>Imagen 20. Generación exitosa de reportes (WF3)</i>	27
<i>Imagen 21. Listado de reportes generados</i>	27
<i>Imagen 22. Visualización del reporte generado</i>	28
<i>Imagen 23. Ejecución exitosa del workflow WF3 – Salida de reportes</i>	28
6.5. PRUEBAS DEL WORKFLOW WF4	29
<i>Imagen 24. Diseño del workflow WF4 – Error Handler</i>	29
<i>Imagen 25. Archivo errors.log generado por WF4</i>	29
<i>Imagen 26. Visualización de errores desde el panel web</i>	30
<i>Imagen 27. Ejecución exitosa del workflow WF4</i>	30
CONCLUSIONES	31
RECOMENDACIONES	32

INTRODUCCIÓN

En este proyecto se desarrolló un sistema automatizado para la gestión de una farmacia utilizando n8n, Docker y PostgreSQL. La idea principal del sistema es ayudar a tener un mejor control del inventario de medicamentos y automatizar procesos que normalmente se realizan de manera manual.

Actualmente, muchas farmacias pequeñas todavía manejan el control de productos de forma manual, lo que puede ocasionar problemas como errores en las existencias, pérdida de información o dificultad para detectar medicamentos con poco stock o próximos a vencer. Debido a esto, se decidió desarrollar una solución automatizada que permitiera mejorar la administración del inventario.

El sistema funciona mediante workflows creados en n8n, los cuales reciben información desde un frontend conectado por medio de API. Después de recibir los datos, el sistema realiza validaciones, actualiza información del inventario y genera alertas automáticas cuando detecta productos con pocas existencias o medicamentos cercanos a su fecha de vencimiento.

Además, el proyecto genera reportes automáticos en formato CSV y cuenta con un sistema de manejo de errores que permite registrar fallos y mantener un mejor control del funcionamiento general del sistema.

Todo el proyecto fue desarrollado en un entorno local utilizando Docker para facilitar la ejecución y PostgreSQL como base de datos principal para almacenar la información.

PLANTEAMIENTO DEL PROBLEMA

En muchas farmacias pequeñas o medianas, el control del inventario todavía se realiza manualmente o utilizando herramientas básicas que no automatizan los procesos importantes. Esto puede generar problemas como descontrol de existencias, falta de medicamentos importantes o dificultad para identificar productos próximos a vencer.

Otro problema frecuente es que los empleados deben revisar constantemente el inventario para verificar qué medicamentos tienen pocas unidades disponibles. Este proceso puede consumir tiempo y aumentar la posibilidad de errores humanos.

Además, cuando no existe un sistema de alertas automáticas, es más difícil reaccionar rápidamente ante productos con bajo stock o medicamentos cercanos a vencer, lo cual puede afectar el funcionamiento y rendimiento de la farmacia.

Por esta razón, se desarrolló un sistema automatizado de gestión de farmacia utilizando n8n, PostgreSQL y Docker, con el objetivo de mejorar el control del inventario, automatizar tareas y generar alertas y reportes de manera automática.

OBJETIVOS

Objetivo General

Desarrollar un sistema automatizado de gestión de farmacia utilizando n8n, Docker y PostgreSQL para mejorar el control del inventario de medicamentos y automatizar procesos relacionados con existencias, alertas y reportes.

Objetivos Específicos

- Automatizar el registro y procesamiento del inventario de medicamentos.
- Validar automáticamente las existencias disponibles dentro del sistema.
- Generar alertas cuando los productos tengan pocas unidades disponibles.
- Detectar medicamentos próximos a vencer.
- Generar reportes automáticos en formato CSV.
- Implementar manejo de errores mediante logs automáticos.
- Utilizar Docker para facilitar la ejecución local del sistema.

1. ARQUITECTURA DEL SISTEMA

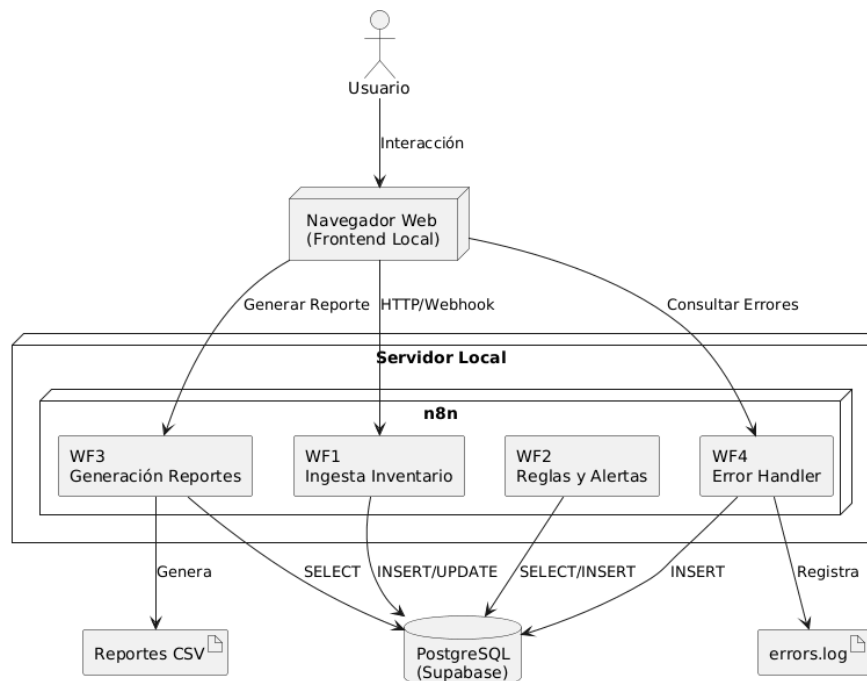
El sistema fue desarrollado utilizando Docker para ejecutar todos los servicios de manera local y organizada dentro de contenedores. Esto permitió facilitar la configuración y ejecución del proyecto sin depender de servicios externos.

La automatización principal fue realizada utilizando n8n, donde se crearon diferentes workflows encargados de procesar información relacionada con el inventario de la farmacia. Los datos son enviados desde el frontend hacia la API de n8n mediante Webhooks.

Después de recibir la información, el sistema realiza validaciones relacionadas con existencias, productos con bajo stock y medicamentos próximos a vencer. Toda la información procesada es almacenada en PostgreSQL para mantener un control del inventario y los registros generados.

Además, el sistema genera alertas automáticas, reportes CSV y registros de errores mediante workflows adicionales. Esto permite mantener un mejor monitoreo del funcionamiento general del sistema y facilitar la administración de la farmacia.

Imagen 1. Diagrama UML de Despliegue de la Arquitectura Local del Sistema



El diagrama UML de despliegue muestra la arquitectura general del Sistema Automatizado de Gestión de Farmacia y la forma en que interactúan sus principales componentes. El usuario accede al sistema mediante un navegador web, donde utiliza el panel desarrollado para registrar movimientos de inventario, consultar alertas, generar reportes y visualizar errores.

Las solicitudes realizadas desde el frontend son enviadas hacia n8n, plataforma encargada de ejecutar los workflows de automatización. Dentro de n8n se encuentran los cuatro procesos principales del sistema: la gestión de inventario (WF1), la generación de alertas (WF2), la creación de reportes (WF3) y el manejo de errores (WF4).

Todos los workflows se encuentran conectados a la base de datos PostgreSQL, utilizada para almacenar la información relacionada con productos, movimientos, alertas, auditorías y errores. Esto permite mantener la información centralizada y disponible para los diferentes procesos automatizados.

Adicionalmente, el sistema genera archivos externos como reportes en formato CSV y archivos de registro (logs), los cuales facilitan el monitoreo, la auditoría y el análisis de la información generada por la aplicación. Esta arquitectura permite automatizar tareas operativas, mejorar el control del inventario y mantener un funcionamiento organizado y eficiente dentro de la farmacia.

2. TECNOLOGIAS USADAS

- **Docker:** fue utilizado para ejecutar todos los servicios del proyecto en contenedores. Esto permitió mantener organizado el entorno de desarrollo y facilitar la ejecución local del sistema.
- **n8n:** fue utilizado para desarrollar los workflows de automatización del sistema. Mediante diferentes nodos se implementaron procesos de validación, generación de alertas, reportes y manejo de errores.
- **PostgreSQL:** fue utilizado como base de datos principal del sistema. En ella se almacena la información relacionada con inventario, movimientos, alertas y registros de errores.
- **Postman:** fue utilizado para realizar pruebas sobre los endpoints de la API conectados a los workflows de n8n. Esto permitió validar el envío de datos y comprobar el correcto funcionamiento del sistema.
- **Frontend Web:** el frontend fue utilizado para permitir la interacción del usuario con el sistema, facilitando el ingreso de información relacionada con medicamentos e inventario.
- **CSV:** el formato CSV fue utilizado para generar reportes automáticos relacionados con el estado del inventario y las alertas generadas por el sistema.

3. DESCRIPCIÓN DE LOS WORKFLOWS

3.1. Workflow 1 – Ingesta de Inventario

El Workflow “WF1_Ingesta_Inventario” fue desarrollado para recibir y procesar información relacionada con movimientos de inventario dentro de la farmacia. Este workflow funciona como el punto principal de entrada de datos al sistema mediante un Webhook conectado a la API de n8n.

El proceso inicia cuando el sistema recibe información desde el frontend o desde pruebas realizadas con Postman. Después de recibir los datos, el workflow normaliza los campos y realiza validaciones para verificar que la información enviada sea correcta antes de continuar con el proceso.

Posteriormente, el sistema identifica el tipo de movimiento que se desea realizar, ya sea una entrada o salida de productos. En caso de ser una entrada, el sistema actualiza el stock y registra el movimiento en PostgreSQL. Si se trata de una salida, primero verifica que existan suficientes unidades disponibles antes de descontar el stock y registrar el movimiento correspondiente.

Además, el workflow incluye manejo de errores y validaciones automáticas. Cuando ocurre un problema relacionado con datos inválidos o falta de existencias, el sistema registra el error automáticamente y genera respuestas controladas para evitar fallos dentro del proceso.

Nodos utilizados

- **Webhook:** recibe las solicitudes enviadas desde el frontend o desde Postman hacia la API de n8n.
- **Set (Normalizar Campos):** organiza y prepara la información recibida para mantener un formato correcto dentro del workflow.
- **IF:** valida condiciones importantes, como verificar que los datos enviados sean válidos o que exista suficiente stock disponible.
- **Switch:** identifica si el movimiento corresponde a una entrada o salida de inventario.
- **PostgreSQL:** realiza consultas y actualizaciones dentro de la base de datos, como registrar movimientos o actualizar stock.
- **Respond to Webhook:** devuelve una respuesta HTTP indicando si el proceso fue realizado correctamente o si ocurrió un error.

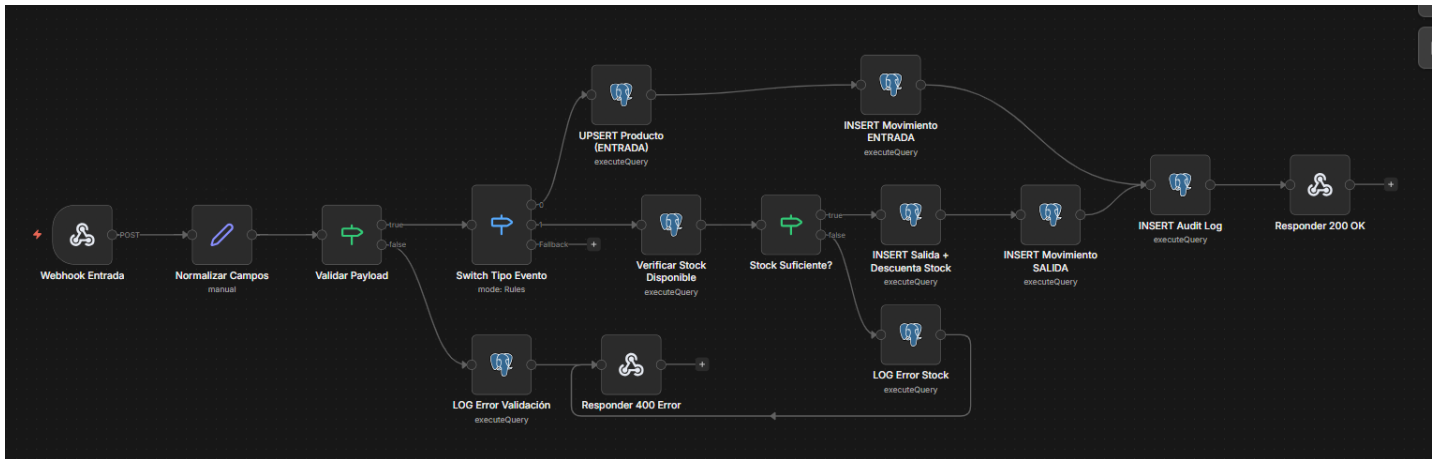


Imagen 2. Workflow de ingesta de inventario en n8n

Explicación general del flujo

1. El workflow inicia cuando el Webhook recibe información enviada desde el frontend o desde Postman. Estos datos contienen información relacionada con movimientos de inventario dentro de la farmacia.
2. Después de recibir la información, el sistema organiza y normaliza los campos para mantener un formato correcto antes de iniciar el procesamiento de los datos.
3. Luego, el workflow valida que la información enviada sea correcta y que los datos importantes no estén vacíos para evitar errores durante la ejecución.
4. El sistema utiliza un nodo Switch para identificar si el movimiento corresponde a una entrada o una salida de productos dentro del inventario.
5. Si se trata de una entrada, el sistema actualiza el stock del medicamento y registra el movimiento en PostgreSQL. Si es una salida, primero verifica que existan suficientes unidades disponibles.
6. Cuando ocurre un error o no hay suficiente stock, el workflow registra automáticamente el problema y genera una respuesta indicando el error encontrado.
7. Finalmente, el sistema guarda un registro de auditoría y devuelve una respuesta HTTP indicando si el proceso fue ejecutado correctamente.

3.2. Workflow 2 – Procesamiento de reglas y alertas

El Workflow “WF2_Procesamiento_Reglas_Alertas” fue desarrollado para monitorear automáticamente el estado del inventario y generar alertas relacionadas con productos con bajo stock o medicamentos próximos a vencer.

Este workflow se ejecuta automáticamente cada cinco minutos mediante un nodo Cron. Después de iniciar, el sistema consulta los productos activos almacenados en PostgreSQL y procesa cada medicamento de forma individual utilizando un nodo Split.

Posteriormente, el workflow verifica si existen productos con pocas existencias disponibles. Cuando el sistema detecta stock bajo, valida que no exista una alerta duplicada antes de registrar una nueva alerta dentro de la base de datos.

Además, el sistema también revisa si los medicamentos tienen fecha de vencimiento próxima. En caso de detectar productos cercanos a vencer, se genera automáticamente una alerta para mantener un mejor control del inventario y evitar pérdidas de medicamentos.

El workflow también registra auditorías automáticas sobre las alertas generadas, permitiendo mantener trazabilidad del funcionamiento del sistema y de las validaciones realizadas.

Nodos utilizados

- **Schedule Trigger (Cron):** ejecuta automáticamente el workflow cada cinco minutos para revisar constantemente el estado del inventario.
- **PostgreSQL:** consulta los productos activos y registra las alertas generadas dentro de la base de datos.
- **Split In Batches:** procesa cada producto individualmente para realizar validaciones separadas sobre stock y vencimientos.
- **IF:** verifica condiciones como stock bajo, fechas de vencimiento próximas o alertas duplicadas.
- **Switch:** ayuda a dirigir el flujo según el tipo de validación o condición encontrada durante el procesamiento de productos y alertas.

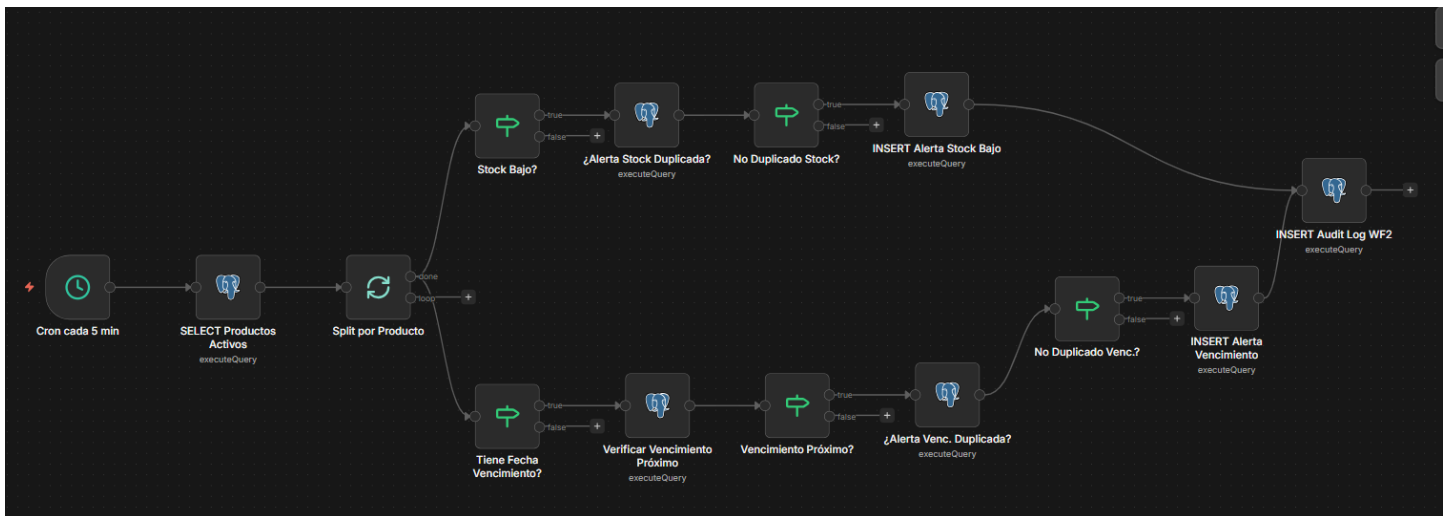


Imagen 3. Workflow de procesamiento de reglas y generación de alertas en n8n

Explicación general del flujo

1. El workflow inicia automáticamente cada cinco minutos utilizando un nodo Cron para revisar constantemente el estado del inventario.
2. El sistema consulta en PostgreSQL todos los productos activos registrados dentro de la farmacia y procesa cada medicamento de forma individual.
3. Después, el workflow verifica si existen productos con pocas existencias disponibles. Si detecta stock bajo, valida que no exista una alerta duplicada antes de registrarla.
4. El sistema también revisa si los medicamentos tienen una fecha de vencimiento próxima para generar alertas preventivas automáticamente.
5. Finalmente, las alertas generadas son almacenadas en PostgreSQL y el workflow registra auditorías para mantener un mejor control del sistema.

3.3. Workflow 3 – Generación de Reportes

El workflow “WF3_Salida_Reportes” fue desarrollado para generar reportes automáticos relacionados con el estado del inventario y las alertas registradas dentro del sistema.

Este workflow puede ejecutarse automáticamente cada semana mediante un nodo Cron o manualmente utilizando un Webhook. Después de iniciar, el sistema consulta en PostgreSQL la información del inventario y las alertas generadas durante la semana.

Posteriormente, los datos son procesados y formateados para generar archivos en diferentes formatos como CSV, HTML y TXT. Luego, el workflow convierte la información a formato binario para permitir la creación de archivos automáticamente.

Dependiendo del tipo de salida, el sistema guarda los reportes dentro de carpetas locales del proyecto. Además, el workflow registra auditorías automáticas relacionadas con la generación de reportes para mantener trazabilidad del sistema.

Nodos utilizados

- **Schedule Trigger (Cron):** permite ejecutar automáticamente la generación de reportes de forma semanal.
- **Webhook:** permite generar reportes manualmente mediante solicitudes enviadas al sistema.
- **PostgreSQL:** consulta la información relacionada con inventario y alertas registradas.
- **Code:** procesa y formatea los datos para generar archivos CSV, HTML y TXT.
- **IF:** identifica el tipo de archivo que debe generarse según el proceso configurado.
- **Write Binary File:** guarda automáticamente los archivos generados dentro de carpetas locales del proyecto.

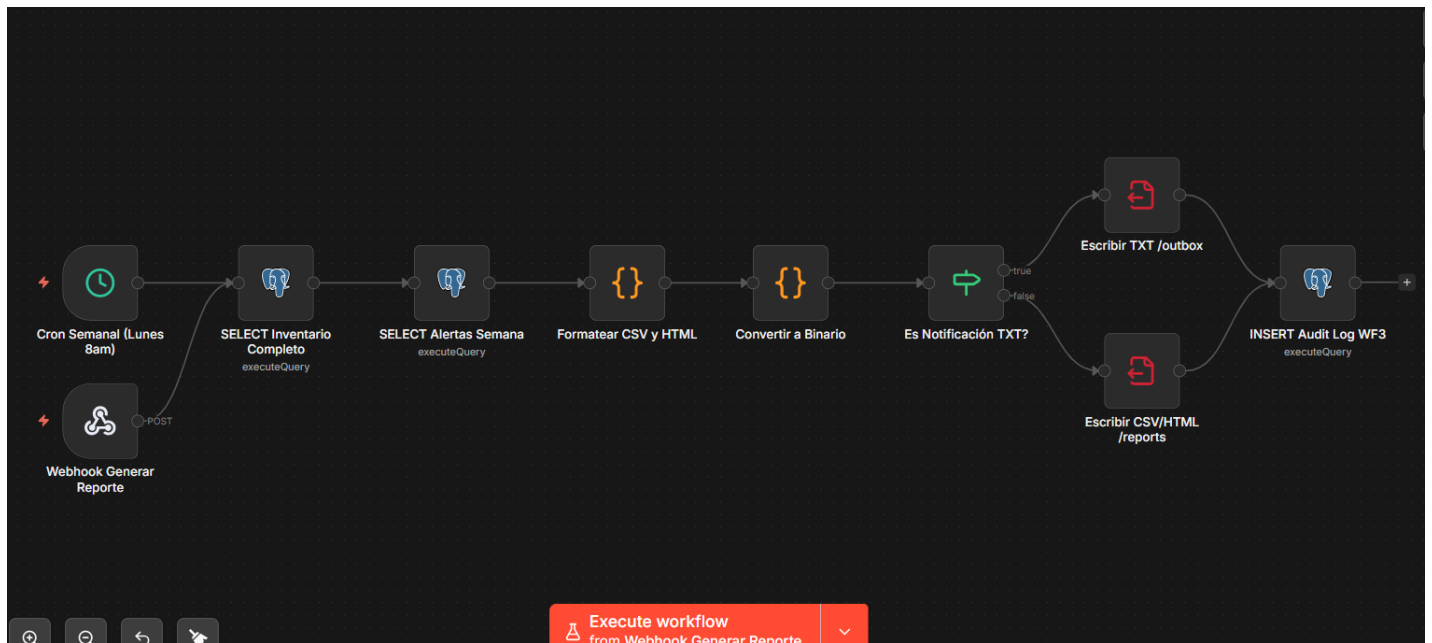


Imagen 4. Workflow de generación de reportes en n8n

Explicación general del flujo

1. El workflow puede ejecutarse automáticamente mediante un Cron semanal o manualmente utilizando un Webhook.
2. El sistema consulta en PostgreSQL la información relacionada con inventario y alertas registradas durante la semana.
3. Después, los datos son procesados y convertidos en formatos como CSV, HTML y TXT para generar reportes automáticos.
4. Dependiendo del tipo de archivo, el workflow guarda los reportes dentro de carpetas locales del sistema.
5. Finalmente, el sistema registra auditorías automáticas relacionadas con la generación de reportes y finaliza el proceso.

3.4. Workflow 4 - Manejo de Errores

El workflow “WF4_Error_Handler” fue desarrollado para detectar y registrar automáticamente los errores generados durante la ejecución de otros workflows dentro del sistema.

Este workflow inicia mediante un nodo Error Trigger, el cual se activa automáticamente cuando ocurre algún fallo en los procesos del sistema. Después de detectar el error, el workflow extrae información importante relacionada con el problema ocurrido.

Posteriormente, el sistema registra la información del error dentro de PostgreSQL y genera alertas automáticas para mantener un mejor control de los fallos detectados. Además, el workflow prepara líneas de log y crea archivos de registro para facilitar el monitoreo del sistema.

Finalmente, el workflow guarda auditorías relacionadas con los errores procesados, permitiendo mantener trazabilidad de los problemas ocurridos durante la ejecución de la automatización.

Nodos utilizados

- **Error Trigger:** detecta automáticamente errores generados dentro de otros workflows del sistema.
- **Code:** procesa la información del error y prepara los datos necesarios para generar logs y registros.
- **PostgreSQL:** almacena errores, alertas y auditorías dentro de la base de datos.
- **Write Binary File:** genera y guarda automáticamente archivos de log relacionados con los errores detectados.

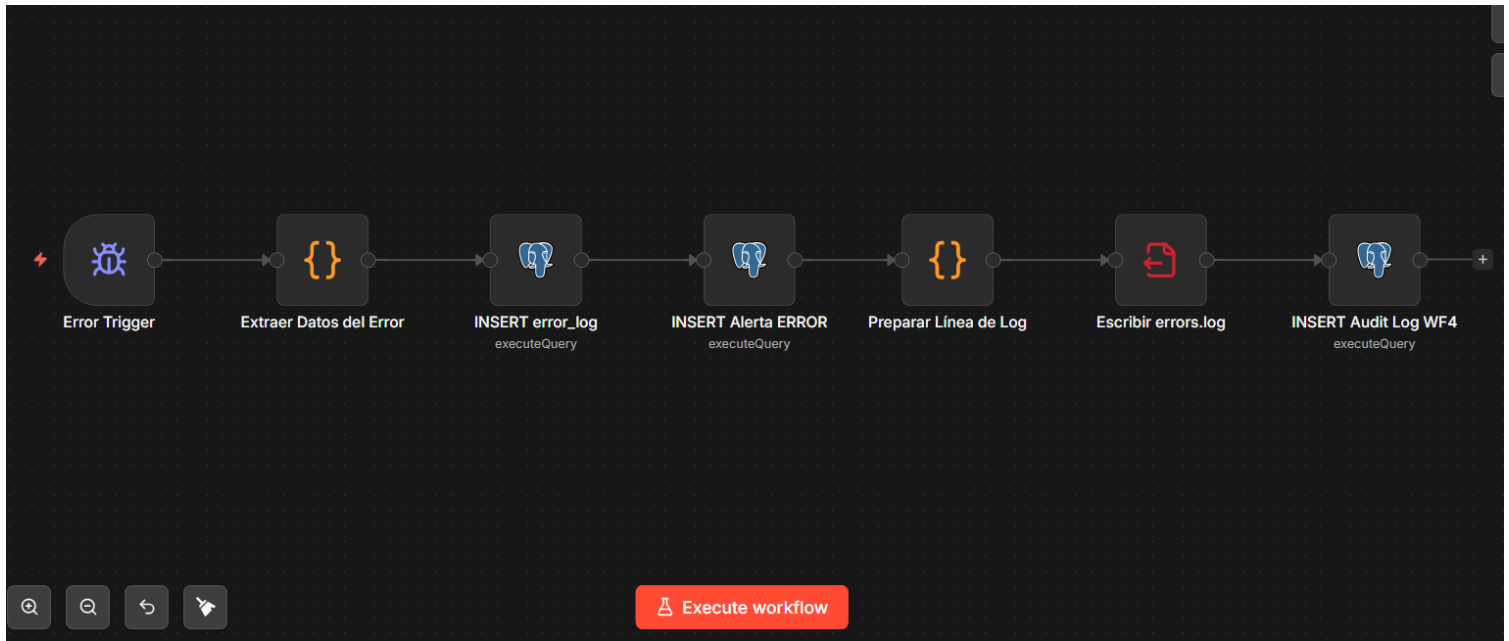


Imagen 5. Workflow de manejo de errores en n8n

Explicación general del flujo

1. El workflow inicia automáticamente cuando ocurre un error dentro de otro workflow del sistema.
2. Después, el sistema extrae información importante relacionada con el error detectado.
3. El workflow registra los errores y alertas dentro de PostgreSQL para mantener control de los fallos ocurridos.
4. Luego, el sistema prepara líneas de log y genera archivos de registro automáticamente.
5. Finalmente, el workflow guarda auditorías relacionadas con los errores procesados y finaliza la ejecución.

4. BASE DE DATOS

La base de datos constituye uno de los componentes principales del Sistema Inteligente de Control de Inventario para Farmacias, ya que permite almacenar y gestionar toda la información relacionada con productos, movimientos de inventario, alertas automáticas, auditorías y registros de errores. Para el desarrollo del proyecto se utilizó PostgreSQL debido a su estabilidad, seguridad y capacidad para manejar grandes volúmenes de información de manera eficiente.

La integración entre PostgreSQL y n8n permite que los workflows desarrollados puedan consultar, insertar y actualizar datos de forma automática. Gracias a esta integración, el sistema puede registrar entradas y salidas de productos, generar alertas por stock bajo o vencimientos próximos, mantener registros históricos y almacenar errores detectados durante la ejecución de los procesos.

Además, se implementaron índices y restricciones que permiten mejorar el rendimiento de las consultas y garantizar la integridad de la información almacenada. Esto ayuda a mantener un control más preciso del inventario y facilita la toma de decisiones dentro de la farmacia.

4.1. Diagrama relacional de la base de datos

La estructura de la base de datos fue diseñada utilizando seis tablas principales relacionadas entre sí. Estas tablas permiten almacenar toda la información necesaria para el funcionamiento del sistema y mantener una correcta organización de los datos.

Las tablas principales son:

- productos
- movimientos
- salidas
- alertas
- audit_log
- error_log

Las relaciones existentes permiten vincular productos con movimientos de inventario, salidas de stock y alertas generadas automáticamente por el sistema.



Imagen 6. Diagrama relacional de la base de datos

El diagrama muestra las relaciones entre las tablas principales del sistema. Se puede observar que la tabla productos es la entidad central, ya que se relaciona con movimientos, salidas y alertas. Por otra parte, las tablas audit_log y error_log funcionan como registros independientes utilizados para almacenar auditorías y errores generados por los workflows.

4.2. Estructura de la tabla productos

La tabla productos almacena la información principal de los medicamentos e insumos registrados dentro del inventario. En ella se guardan datos como código SKU, nombre del producto, categoría, cantidad disponible, stock mínimo y fecha de vencimiento.

Imagen 7. Estructura de la tabla productos

```

-- TABLA: productos
-- =====
CREATE TABLE IF NOT EXISTS public.productos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sku TEXT UNIQUE NOT NULL,
    nombre TEXT NOT NULL,
    categoria TEXT NOT NULL,
    stock_actual INTEGER NOT NULL DEFAULT 0,
    stock_minimo INTEGER NOT NULL DEFAULT 0,
    fecha_vencimiento DATE NULL,
    activo BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- Índices para productos
CREATE INDEX IF NOT EXISTS idx_productos_sku ON public.productos(sku);
CREATE INDEX IF NOT EXISTS idx_productos_activo ON public.productos(activo);
CREATE INDEX IF NOT EXISTS idx_productos_fecha_vencimiento ON public.productos(fecha_vencimiento);
  
```

Esta tabla es utilizada constantemente por los workflows para consultar existencias, actualizar cantidades y verificar condiciones relacionadas con stock mínimo y fechas de vencimiento.

4.3. Estructura de la tabla alertas

La tabla alertas almacena todas las notificaciones generadas automáticamente por el sistema. Estas alertas permiten informar al usuario sobre situaciones que requieren atención, como productos con bajo inventario, medicamentos próximos a vencer o errores detectados durante la ejecución de los procesos.

Imagen 8. Estructura de la tabla alertas

```
-- TABLA: alertas
-- =====
CREATE TABLE IF NOT EXISTS public.alertas (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tipo TEXT NOT NULL CHECK (tipo IN ('STOCK_BAJO', 'VENCIMIENTO_PROXIMO', 'ERROR')),
  nivel TEXT NOT NULL CHECK (nivel IN ('INFO', 'WARNING', 'CRITICAL')),
  mensaje TEXT NOT NULL,
  producto_id UUID NULL REFERENCES public.productos(id) ON DELETE CASCADE,
  salida_id UUID NULL REFERENCES public.salidas(id) ON DELETE CASCADE,
  resuelta BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

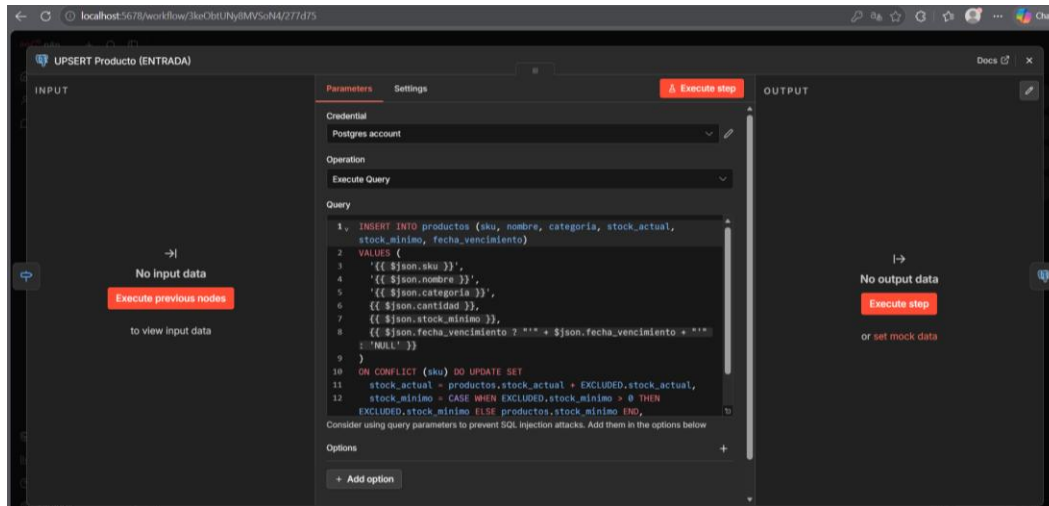
-- Índices para alertas
CREATE INDEX IF NOT EXISTS idx_alertas_tipo ON public.alertas(tipo);
CREATE INDEX IF NOT EXISTS idx_alertas_nivel ON public.alertas(nivel);
CREATE INDEX IF NOT EXISTS idx_alertas_producto_id ON public.alertas(producto_id);
CREATE INDEX IF NOT EXISTS idx_alertas_salida_id ON public.alertas(salida_id);
CREATE INDEX IF NOT EXISTS idx_alertas_resuelta ON public.alertas(resuelta);
CREATE INDEX IF NOT EXISTS idx_alertas_created_at ON public.alertas(created_at);
```

La implementación de esta tabla permite mantener un monitoreo constante del estado del inventario y facilita la toma de decisiones preventivas dentro de la farmacia.

4.4. Integración de PostgreSQL con n8n

La comunicación entre la base de datos y los workflows se realiza mediante nodos PostgreSQL configurados dentro de n8n. Estos nodos ejecutan consultas SQL que permiten insertar, actualizar y consultar información automáticamente según los eventos recibidos por el sistema.

Imagen 9. Nodo PostgreSQL utilizado en n8n



En la imagen anterior se observa una consulta SQL utilizada para registrar nuevos productos o actualizar información existente dentro del inventario. Este proceso permite mantener la base de datos sincronizada con los datos recibidos desde el frontend y los webhooks del sistema.

4.5. Datos de prueba utilizados

Para validar el correcto funcionamiento del sistema se implementó un conjunto de datos de prueba (Seed Data). Estos registros simulan situaciones reales que pueden presentarse en una farmacia, como productos con stock bajo, medicamentos próximos a vencer y diferentes categorías de inventario.

Imagen 10. Datos de prueba utilizados por el sistema

```
-- A) PRODUCTOS
-- =====
INSERT INTO public.productos (id, sku, nombre, categoria, stock_actual, stock_minimo, fecha_vencimiento, activo)
VALUES
('a1000000-0000-0000-0000-000000000001', 'AMOX-500', 'Amoxicilina 500mg (caja 20)', 'Antibióticos', 150, 50, '2027-06-30', true),
('a1000000-0000-0000-0000-000000000002', 'IBUP-400', 'Ibuprofeno 400mg (caja 30)', 'Analgésicos', 200, 80, '2026-12-31', true),
('a1000000-0000-0000-0000-000000000003', 'PARA-500', 'Paracetamol 500mg (caja 50)', 'Analgésicos', 30, 40, '2026-04-15', true), -- stock bajo
('a1000000-0000-0000-0000-000000000004', 'METF-850', 'Metformina 850mg (caja 30)', 'Antidiabéticos', 90, 30, '2027-01-15', true),
('a1000000-0000-0000-0000-000000000005', 'LORAT-10', 'Loratadina 10mg (caja 20)', 'Antialérgicos', 60, 25, NULL, true),
('a1000000-0000-0000-0000-000000000006', 'VENC-TEST', 'Producto próximo a vencer', 'Prueba', 20, 5, '2026-03-10', true), -- stock bajo + vencimie
('a1000000-0000-0000-0000-000000000007', 'ALCOHOL-70', 'Alcohol isopropílico 70% (1L)', 'Insumos', 80, 20, '2026-09-30', true),
('a1000000-0000-0000-0000-000000000008', 'JERINGAS-5', 'Jeringas 5ml (caja 100)', 'Insumos', 500, 100, NULL, true);
```

Los datos de prueba permitieron verificar el funcionamiento de los workflows, la generación automática de alertas, la actualización de existencias y la creación de reportes dentro del sistema.

En general, la base de datos desarrollada proporciona una estructura sólida para la gestión automatizada del inventario, permitiendo almacenar información de forma organizada, mantener la integridad de los datos y facilitar la automatización de procesos mediante la integración con n8n.

5. CONSIDERACIONES DE SEGURIDAD

La seguridad es un aspecto importante dentro del Sistema Automatizado de Gestión de Farmacia, ya que permite proteger la información almacenada y garantizar el correcto funcionamiento de los procesos implementados. Para ello, se aplicaron validaciones sobre los datos recibidos desde la interfaz web antes de ser procesados por los workflows de n8n, reduciendo la posibilidad de errores y registros inconsistentes en la base de datos.

El sistema utiliza PostgreSQL como motor de almacenamiento principal, permitiendo gestionar de forma organizada la información relacionada con productos, movimientos, alertas y registros de auditoría. Asimismo, se implementó la tabla **audit_log**, la cual registra las acciones ejecutadas por los workflows y facilita el monitoreo de las operaciones realizadas dentro de la aplicación. Esta bitácora proporciona trazabilidad y permite verificar eventos ocurridos durante la ejecución del sistema.

Adicionalmente, se desarrolló el workflow **WF4 – Error Handler**, encargado de detectar y registrar errores generados durante la ejecución de los procesos automatizados. Los incidentes son almacenados en la base de datos y en archivos de log para facilitar su análisis y corrección. Finalmente, la utilización de Docker permitió ejecutar los servicios en un entorno controlado y organizado, contribuyendo a la estabilidad, mantenimiento y administración general de la solución desarrollada.

Otro aspecto importante considerado durante el desarrollo fue la separación de funciones entre los distintos workflows implementados. Cada flujo fue diseñado para realizar tareas específicas, como el manejo de inventario, la generación de alertas, la creación de reportes o la gestión de errores. Esta división permite reducir el impacto de posibles fallos, facilita el mantenimiento del sistema y mejora la organización de los procesos automatizados.

Finalmente, la generación automática de reportes y registros de eventos contribuye a mantener un mayor control sobre la información procesada por la aplicación. Gracias a estos mecanismos es posible verificar el historial de operaciones realizadas, identificar incidencias de manera más rápida y disponer de evidencia sobre el funcionamiento del sistema. Estas medidas fortalecen la confiabilidad de la solución y favorecen una administración más eficiente del inventario de la farmacia.

6. EVIDENCIAS DE EJECUCIÓN Y RESULTADOS DEL SISTEMA

6.1. Panel Principal del Sistema

Imagen 11. Panel principal de pruebas WF1-WF4

SISTEMA DE INVENTARIO INTELIGENTE

Panel de Pruebas WF1-WF4

Controla ingesta, revisa ejecuciones cron y monitorea reportes/errores de tus workflows en n8n

Configuracion

URL base n8n

Endpoint

http://localhost:5678

/webhook/inventory/intake

WF1 - Ingesta Inventario

Entrada

Salida

SKU

Cantidad

TEST001

1

Nombre

Categoria

Producto Test

General

Stock minimo

Fecha vencimiento

0

dd/mm/aaaa

Enviar evento

Cargar ejemplo

Respuesta

Listo para enviar

```
{}
```

WF2 - Reglas y Alertas (Cron)

Actualizar estado

WF2 corre automaticamente cada 5 minutos desde n8n. Aqui validas estado del panel y ejecutas chequeos rapidos.

```
{
  "ok": true,
  "timestamp": "2026-05-29T05:46:04.257Z",
  "notes": "WF2 se ejecuta por cron cada 5 minutos en n8n"
}
```

WF3 - Reportes Generados

Generar reporte ahora

Recargar reportes

Endpoint WF3 manual

/webhook/reports/generate

reporte_inventario_2026-05-14_20260513_232948.csv

14/5/2026, 1:29:48 p. m. | 1074 bytes

Abrir

reporte_inventario_2026-05-14_20260513_232219.csv

14/5/2026, 1:22:19 p. m. | 1013 bytes

Abrir

reporte_inventario_2026-05-14_20260513_232148.csv

14/5/2026, 1:21:48 p. m. | 1013 bytes

Abrir

reporte_inventario_2026-05-14_20260513_231737.csv

14/5/2026, 1:17:37 p. m. | 952 bytes

Abrir

reporte_inventario_2026-05-14_20260513_231627.csv

14/5/2026, 1:16:27 p. m. | 896 bytes

Abrir

reporte_inventario_2026-05-14_20260513_231625.csv

14/5/2026, 1:16:25 p. m. | 896 bytes

Abrir

WF4 - Error Handler

Recargar errores

```
[2026-03-19T05:09:21.893Z] ERROR en "Example Workflow" | Nodo: "Node With Error" | Msg: Sin mensaje de error | ExecID: 231
```

Se muestra el panel web desarrollado para interactuar con los cuatro flujos de trabajo implementados en n8n. Desde esta interfaz es posible registrar movimientos de inventario, consultar alertas automáticas, generar reportes y visualizar errores registrados por el sistema.

6.2. Pruebas del Workflow WF1

Imagen 12. Registro exitoso de ingreso de inventario (WF1)

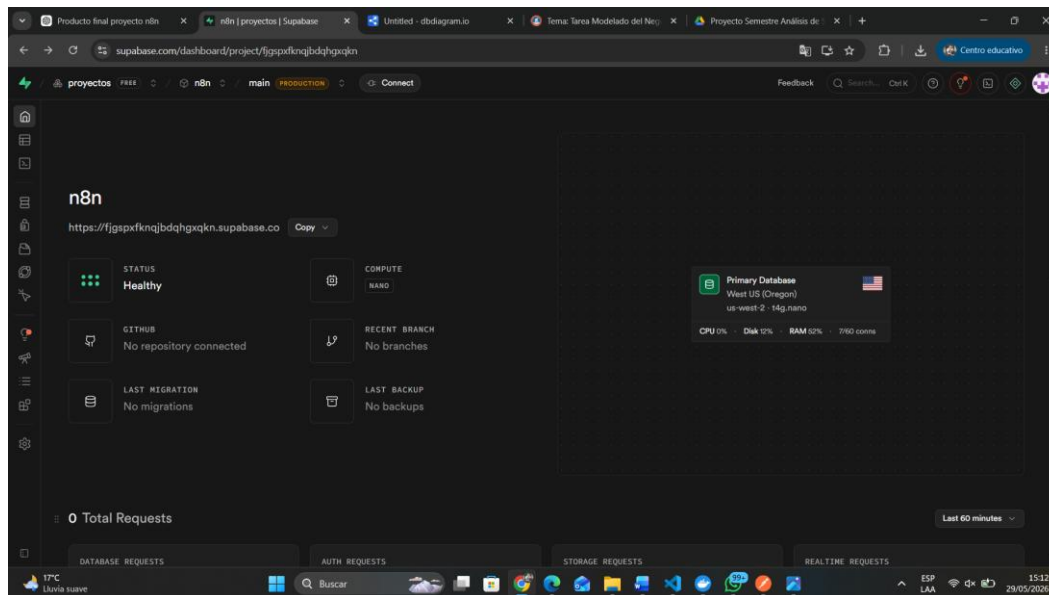
The screenshot shows a web application interface for inventory management. The form is titled 'WF1 - Ingesta Inventario' and includes fields for SKU, Cantidad, Nombre, Categoria, Stock minimo, and Fecha vencimiento. The 'Entrada' button is highlighted in green. Below the form, a 'Respuesta' section displays a green message 'Evento procesado correctamente' and a JSON response indicating successful registration.

Field	Value
SKU	Prueba0001
Cantidad	500
Nombre	Amoxicilina
Categoria	Antibioticos
Stock minimo	25
Fecha vencimiento	10/06/2026

```
{
  "ok": true,
  "status": 200,
  "body": {
    "ok": true,
    "tipo_evento": "ENTRADA",
    "sku": "PRUEBA0001",
    "mensaje": "Operaci3n registrada correctamente"
  }
}
```

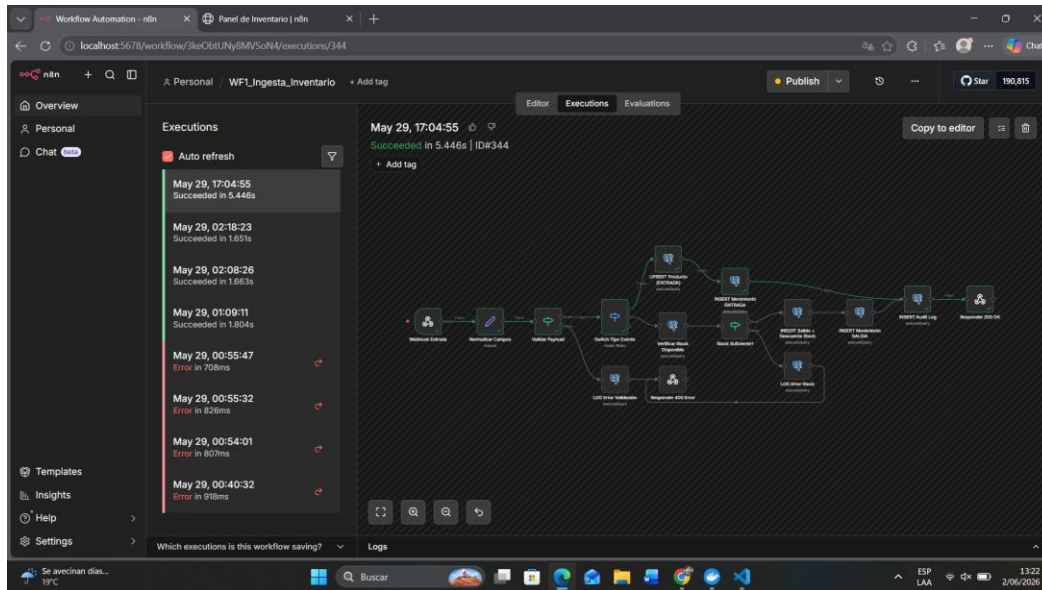
En esta imagen se observa el env3o de un producto mediante el formulario de ingreso de inventario. El sistema procesa correctamente la solicitud y devuelve una respuesta exitosa indicando que la operaci3n fue registrada satisfactoriamente.

Imagen 13. Configuraci3n del proyecto en Supabase



Se muestra el entorno de Supabase utilizado como base de datos principal del sistema. Desde este panel se administra la infraestructura, el almacenamiento de datos y los recursos utilizados por la aplicaci3n.

Imagen 14. Ejecución exitosa del workflow WF1 – Ingesta de Inventario



En esta imagen se muestra la ejecución correcta del workflow WF1 dentro de n8n. El flujo procesa la información recibida desde el panel web, valida los datos ingresados, actualiza el inventario en la base de datos, registra el movimiento correspondiente y genera una entrada en la bitácora de auditoría. La ejecución finaliza exitosamente, confirmando el correcto funcionamiento del proceso de ingreso de productos.

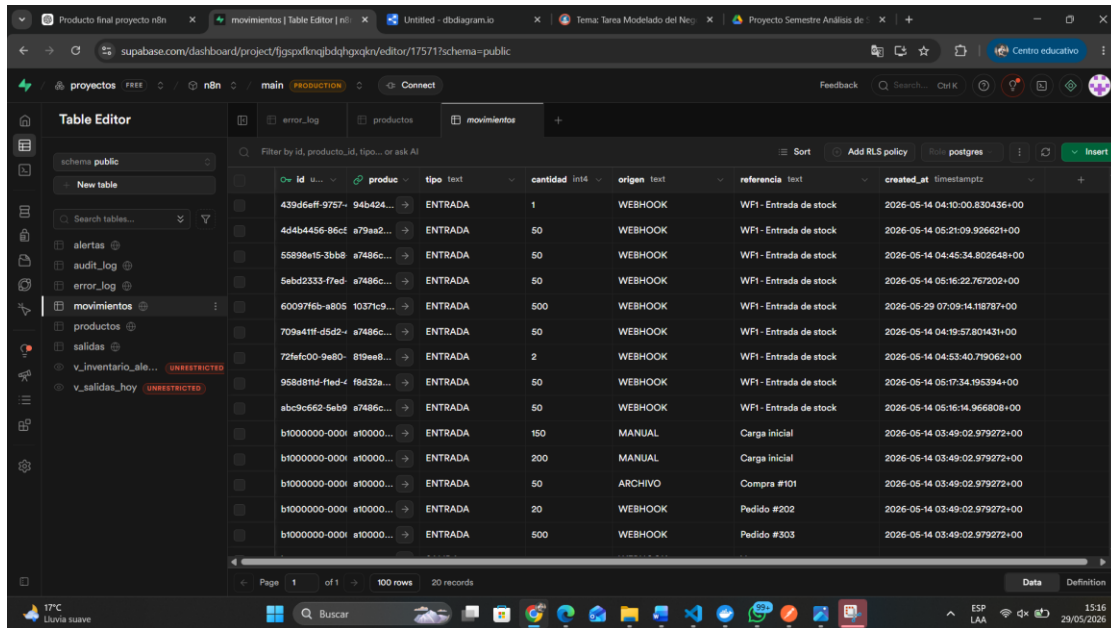
Imagen 15. Tabla Productos en Supabase

The screenshot shows the Supabase Table Editor interface. The 'productos' table is selected, and its schema is displayed. The table has columns: id, sku, nombre, categoria, stock_actual, and stock_minimo. The data is shown in a grid format, with rows representing different products. The interface includes a sidebar with navigation links for 'proyectos', 'main', and 'productos'. The bottom of the interface shows a Windows taskbar with various application icons and the system clock indicating 15:14 on 28/05/2026.

id	sku	nombre	categoria	stock_actual	stock_minimo
0af8ca3d-75a5-MED0005		Paracetamol	Medicamento	50	2
1037c9d4-cd60-PRUEBA0001		Amoxicilina	Antibióticos	500	25
819ee88a-7a83-00001		Laptop	Electronica	2	1
94b4242d-4694-ESDFSD		Prueba	General	1	6
a1000000-000X-AMOX-500		Amoxicilina 500mg (caja 20)	Antibióticos	150	50
a1000000-000X-IBUP-400		Ibuprofeno 400mg (caja 30)	Analgésicos	200	80
a1000000-000X-PARA-500		Paracetamol 500mg (caja 50)	Analgésicos	30	40
a1000000-000X-METF-850		Metformina 850mg (caja 30)	Antidiabéticos	90	30
a1000000-000X-LORAT-10		Loratadina 10mg (caja 20)	Antialérgicos	60	25
a1000000-000X-VENC-TEST		Producto próximo a vencer	Prueba	20	5
a1000000-000X-ALCOHOL-70		Alcohol isopropílico 70% (L)	Insumos	80	20
a1000000-000X-JERINGAS-5		Jeringas 5ml (caja 100)	Insumos	500	100
a7486c57-8b6f-TEST001		Producto Test	Prueba	300	10
a79aa294-5e35-MED0004		Ashwagandha	Medicamento	50	2

Se visualizan los registros almacenados en la tabla productos. Aquí se mantiene la información de cada artículo, incluyendo SKU, nombre, categoría, stock actual y stock mínimo.

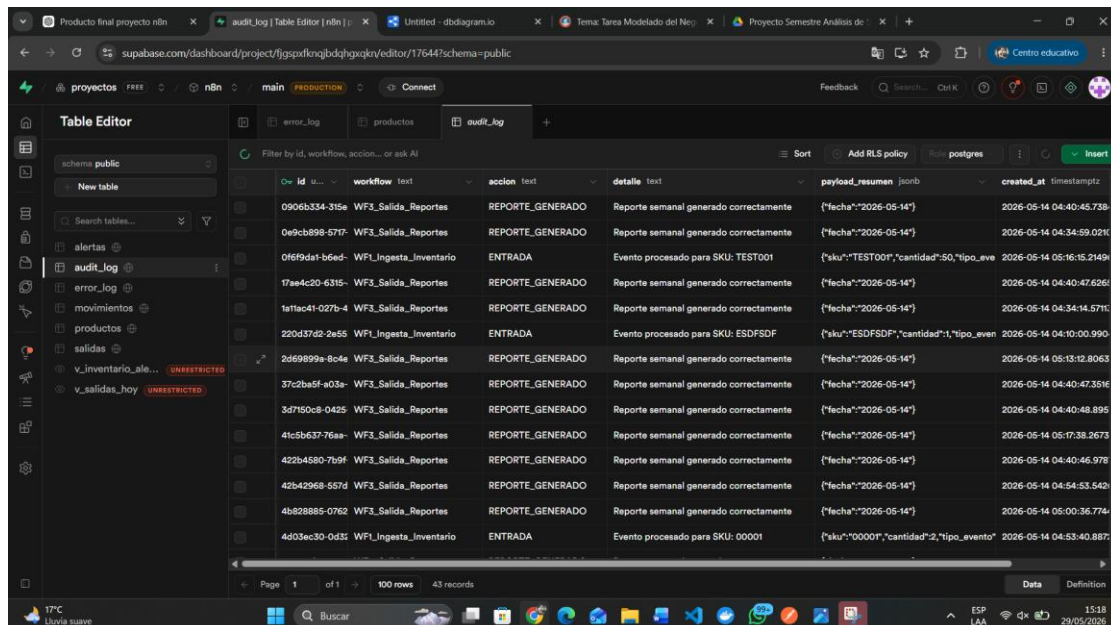
Imagen 16. Tabla Movimientos en Supabase



id	producto_id	tipo	cantidad	origen	referencia	created_at
439d6eff-9757-94b424...	94b424...	ENTRADA	1	WEBHOOK	WFI - Entrada de stock	2026-05-14 04:10:00.830436+00
4d4b4456-86cf-a79aa2...	a79aa2...	ENTRADA	50	WEBHOOK	WFI - Entrada de stock	2026-05-14 05:21:09.926621+00
5589ae15-3bb8-a7486c...	a7486c...	ENTRADA	50	WEBHOOK	WFI - Entrada de stock	2026-05-14 04:45:34.802648+00
5ebd2333-f7ed-a7486c...	a7486c...	ENTRADA	50	WEBHOOK	WFI - Entrada de stock	2026-05-14 05:16:22.767202+00
60097f6b-a805-10371c9...	10371c9...	ENTRADA	500	WEBHOOK	WFI - Entrada de stock	2026-05-29 07:09:14.118787+00
709e41ff-d5d2-a7486c...	a7486c...	ENTRADA	50	WEBHOOK	WFI - Entrada de stock	2026-05-14 04:19:57.801431+00
72f4fc00-9e80-b19ee8...	b19ee8...	ENTRADA	2	WEBHOOK	WFI - Entrada de stock	2026-05-14 04:53:40.719062+00
958d81fd-f1ed-fbd32a...	fbd32a...	ENTRADA	50	WEBHOOK	WFI - Entrada de stock	2026-05-14 05:17:34.195394+00
abc9c662-5eb9-a7486c...	a7486c...	ENTRADA	50	WEBHOOK	WFI - Entrada de stock	2026-05-14 05:16:14.966808+00
b1000000-0001-a10000...	a10000...	ENTRADA	150	MANUAL	Carga inicial	2026-05-14 03:49:02.979272+00
b1000000-0001-a10000...	a10000...	ENTRADA	200	MANUAL	Carga inicial	2026-05-14 03:49:02.979272+00
b1000000-0001-a10000...	a10000...	ENTRADA	50	ARCHIVO	Compra #101	2026-05-14 03:49:02.979272+00
b1000000-0001-a10000...	a10000...	ENTRADA	20	WEBHOOK	Pedido #202	2026-05-14 03:49:02.979272+00
b1000000-0001-a10000...	a10000...	ENTRADA	500	WEBHOOK	Pedido #303	2026-05-14 03:49:02.979272+00

La tabla movimientos almacena el historial de entradas y salidas realizadas sobre los productos. Cada registro conserva la información necesaria para la trazabilidad de las operaciones ejecutadas.

Imagen 17. Tabla Audit Log

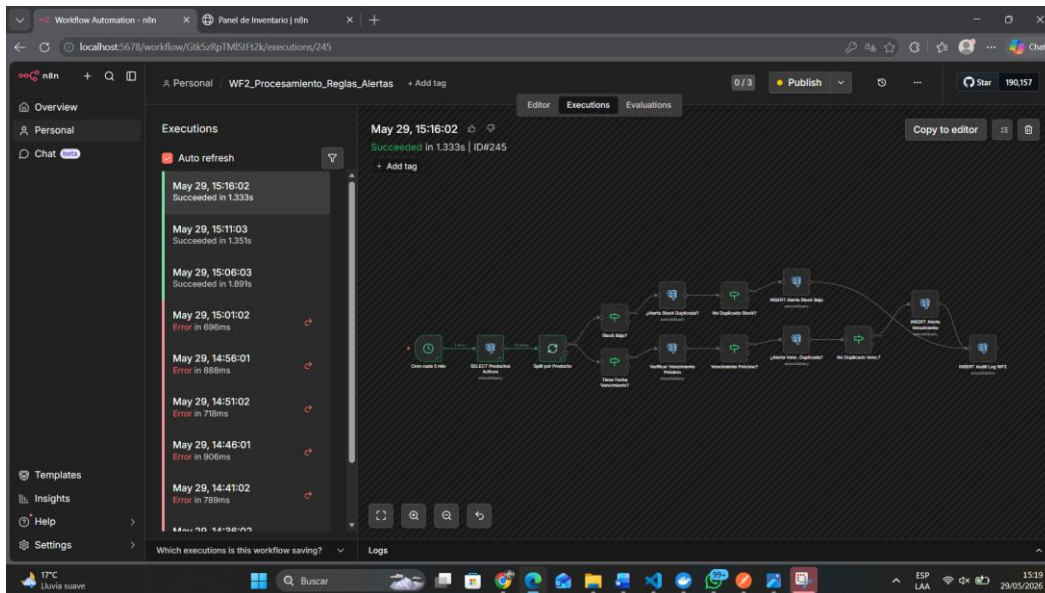


id	workflow	accion	detalle	payload_resumen	created_at
0906b334-315e-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:40:45.738
0e9cb898-57f7-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:34:59.021
0f6f9daf-b6e4-WFI_Ingesta_Inventario	WFI_Ingesta_Inventario	ENTRADA	Evento procesado para SKU: TEST001	("sku": "TEST001", "cantidad": 50, "tipo_evento": "ENTRADA")	2026-05-14 05:16:15.2149
17ae4c20-6315-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:40:47.626
1a1fac41-027b-4-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:34:14.571
220d37d2-2e55-WFI_Ingesta_Inventario	WFI_Ingesta_Inventario	ENTRADA	Evento procesado para SKU: ESDFSDF	("sku": "ESDFSDF", "cantidad": 1, "tipo_evento": "ENTRADA")	2026-05-14 04:10:00.990
2d69899a-8c4e-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 05:13:12.8063
37c2ba5f-a03a-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:40:47.3516
3d7f50c8-0425-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:40:48.895
41c5b637-76aa-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 05:17:38.2673
42b4680-7b9f-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:40:46.978
42b42968-557d-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 04:54:53.542
4b828865-0762-WF3_Salida_Reportes	WFI_Salida_Reportes	REPORTE_GENERADO	Reporte semanal generado correctamente	("fecha": "2026-05-14")	2026-05-14 05:00:36.774
4d03ec30-0d31-WFI_Ingesta_Inventario	WFI_Ingesta_Inventario	ENTRADA	Evento procesado para SKU: 00001	("sku": "00001", "cantidad": 2, "tipo_evento": "ENTRADA")	2026-05-14 04:53:40.887

En esta imagen se observa la bitácora de auditoría del sistema. Cada evento ejecutado por los workflows queda registrado con información detallada sobre la acción realizada, fecha y datos procesados.

6.3. Pruebas del Workflow WF2

Imagen 18. Ejecución exitosa del workflow WF2



Se muestra la ejecución correcta del flujo encargado de analizar el inventario y detectar condiciones especiales como productos con stock bajo o próximos a vencer.

Imagen 19. Alertas generadas automáticamente

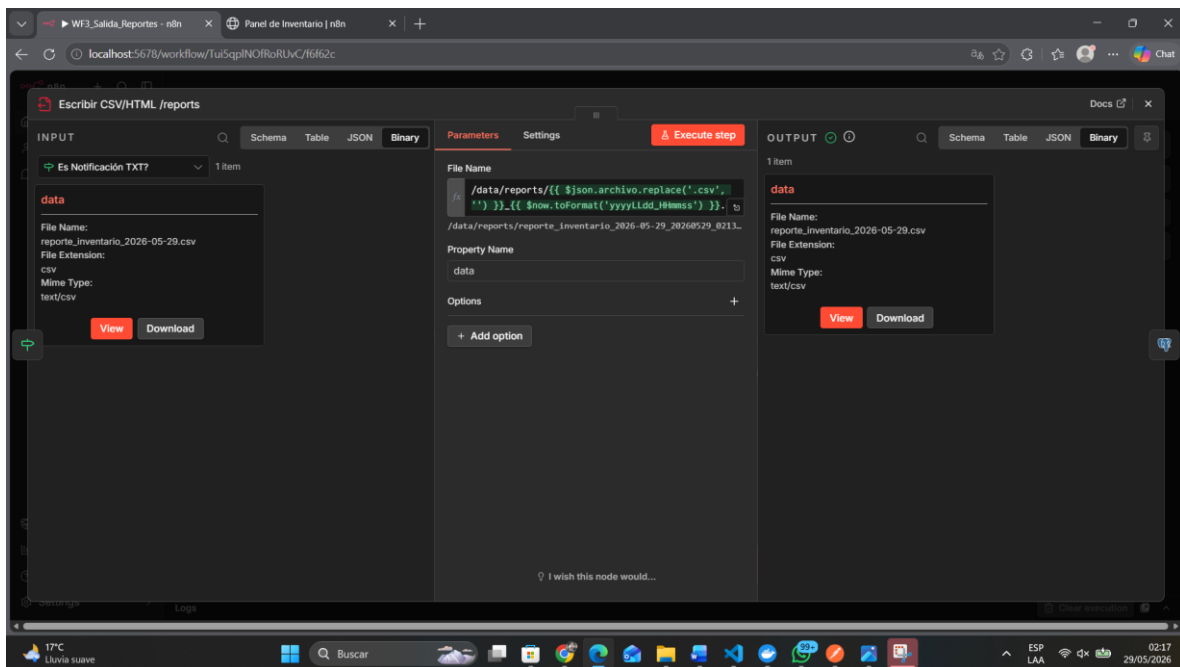
The screenshot shows the Supabase Table Editor interface. The 'alertas' table is selected, and its contents are displayed in a table view. The table has columns for 'id', 'tipo', 'nivel', 'mensaje', and 'producto_id'. The data includes alerts for low stock and upcoming expiration dates for products like Paracetamol and Ibuprofen.

id	tipo	nivel	mensaje	producto_id
d1000000-0001	STOCK_BAJO	CRITICAL	Stock de Paracetamol 600mg por debajo del mínimo. Actual: 30 Mínimo: 40	a10000000-000C
d1000000-0001	STOCK_BAJO	WARNING	Stock bajo de Producto próximo a vencer. Actual: 20 Mínimo: 5	a10000000-000C
d1000000-0001	VENCIMIENTO_PROXIMO	CRITICAL	Producto próximo a vencer vence en 7 días (2026-03-10). Revisar existencias.	a10000000-000C
d1000000-0001	VENCIMIENTO_PROXIMO	INFO	Paracetamol 500mg vence en 43 días (2026-04-15).	a10000000-000C
d1000000-0001	STOCK_BAJO	WARNING	Stock bajo de Ibuprofeno 400mg (resuelto tras reposición).	a10000000-000C

La tabla alertas almacena las notificaciones producidas por el sistema. Entre ellas se encuentran alertas de stock bajo y vencimiento próximo, permitiendo una gestión preventiva del inventario.

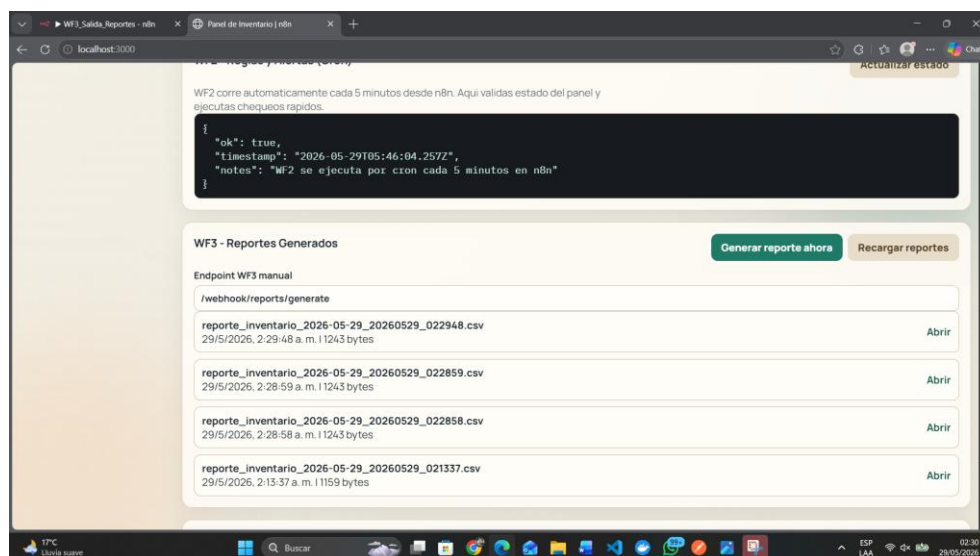
6.4. Pruebas del Workflow WF3

Imagen 20. Generación exitosa de reportes (WF3)



Se observa la correcta ejecución del nodo encargado de escribir archivos CSV dentro de la carpeta de reportes. El workflow completa el proceso sin errores y genera el archivo correspondiente.

Imagen 21. Listado de reportes generados



El panel web muestra los reportes disponibles para consulta. Cada archivo generado contiene la fecha y hora de creación, facilitando el control histórico de la información exportada.

Imagen 22. Visualización del reporte generado

WF2 corre automáticamente cada 5 minutos desde n8n. Aquí validas estado del panel y ejecutas chequeos rápidos.

Descargas

reporte_inventario_2026-05-29_20260529_022948 (1).csv

Abre archivo

Autoguardado

reporte_inventario_2026-05-29_20260529_022948 (1) • Guardado en Este PC

Archivo Inicio Insertar Dibujar Disposición de página Fórmulas Datos Revisar Vista Automatizar Ayuda

Formato condicional Dar formato como tabla Estilos de celda

Eliminar Eliminar Celdas Edición Complementos

Comentarios Compartir

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

Complementos

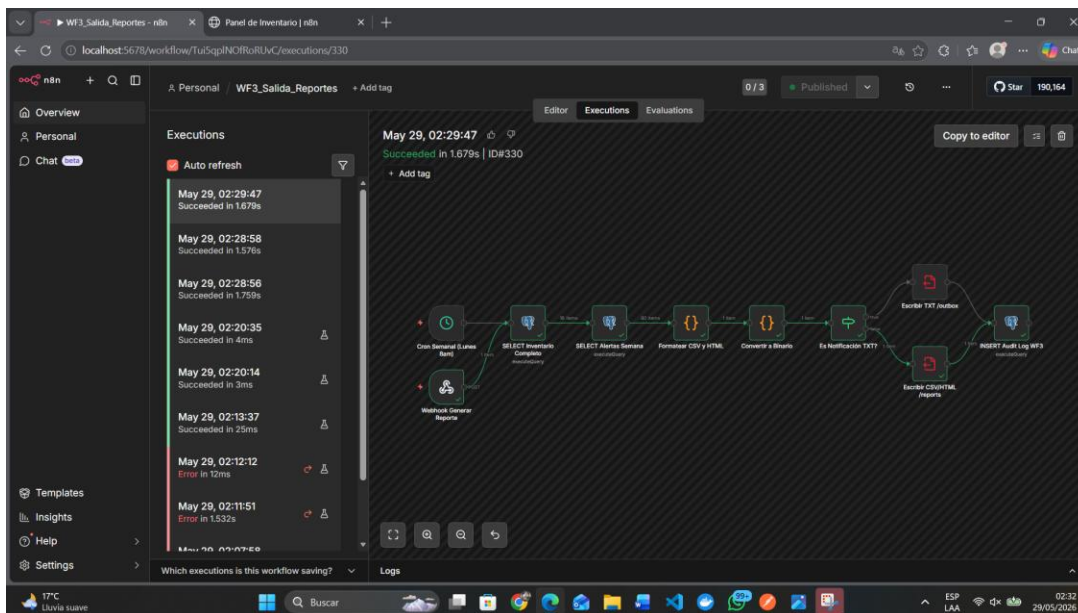
Complementos

Complementos

Complementos

Se abre el archivo CSV exportado por el sistema. El reporte contiene el inventario completo con información como SKU, nombre, categoría, stock actual, stock mínimo, fecha de vencimiento y estado del producto.

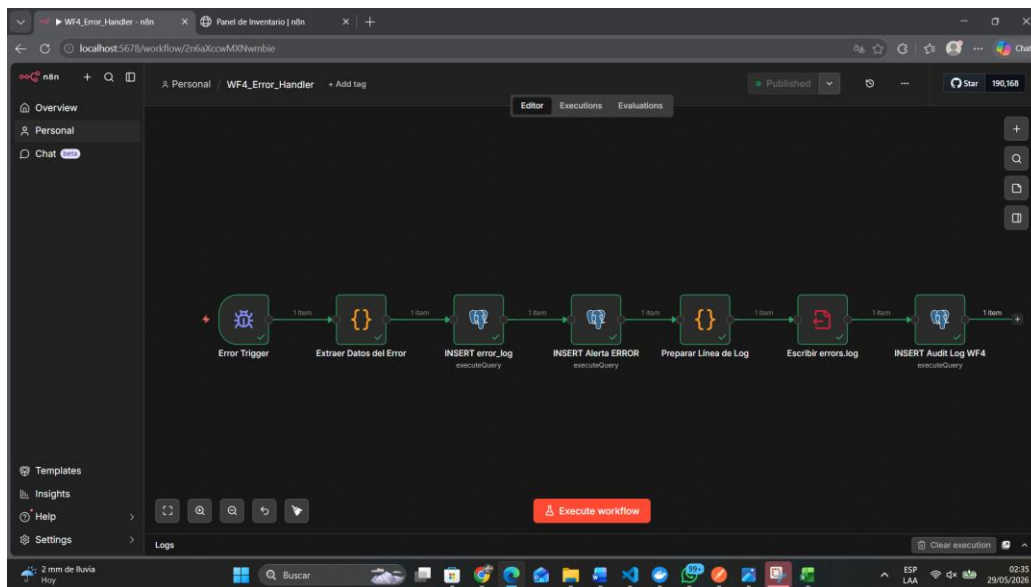
Imagen 23. Ejecución exitosa del workflow WF3 – Salida de reportes



En esta imagen se muestra la ejecución completa del workflow WF3 dentro de n8n. El flujo consulta el inventario, obtiene las alertas registradas, formatea la información en archivo CSV, escribe el reporte en la carpeta correspondiente y registra la acción en la bitácora de auditoría. La ejecución finaliza correctamente, comprobando que la generación automática de reportes funciona de forma adecuada.

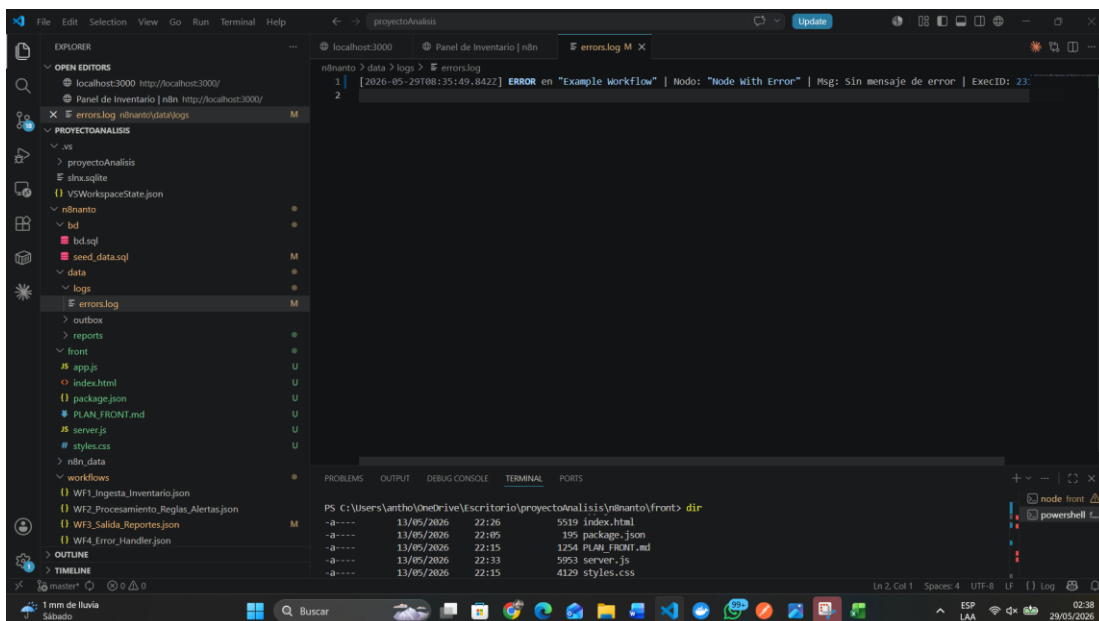
6.5. Pruebas del Workflow WF4

Imagen 24. Diseño del workflow WF4 – Error Handler



Se presenta la estructura completa del flujo encargado de gestionar errores. Este workflow captura excepciones, registra información relevante y genera evidencias para facilitar la identificación de problemas.

Imagen 25. Archivo errors.log generado por WF4



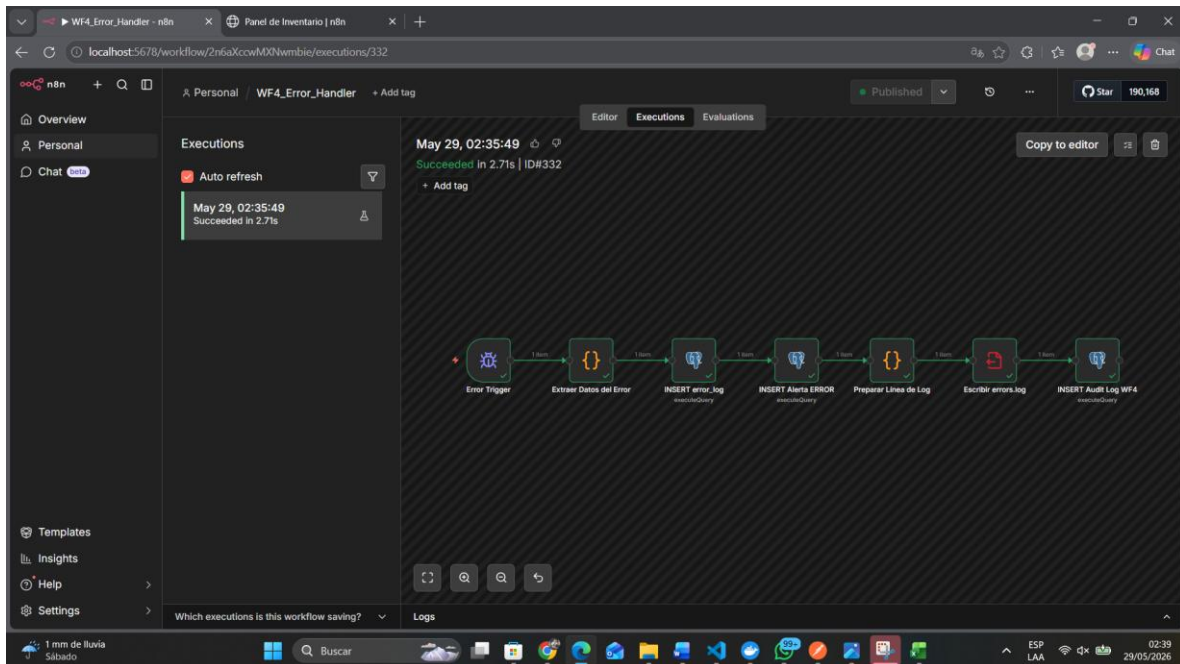
Se visualiza el archivo de registro de errores almacenado por el sistema. Cada línea contiene información relacionada con fallos detectados durante la ejecución de los workflows.

Imagen 26. Visualización de errores desde el panel web



El panel permite consultar los errores registrados sin necesidad de acceder directamente a los archivos internos del sistema, facilitando el monitoreo y la supervisión.

Imagen 27. Ejecución exitosa del workflow WF4



Finalmente se muestra la ejecución satisfactoria del manejador de errores, verificando que el flujo registra adecuadamente los eventos y almacena la información correspondiente en la bitácora del sistema.

CONCLUSIONES

1. Se logró desarrollar e implementar un Sistema Automatizado de Gestión de Farmacia utilizando n8n, PostgreSQL y Docker, permitiendo automatizar procesos relacionados con el control de inventario, generación de alertas, elaboración de reportes y manejo de errores.
2. La utilización de workflows automatizados facilitó la gestión de medicamentos dentro de la farmacia, reduciendo la intervención manual en tareas repetitivas y mejorando la eficiencia en el control de existencias y movimientos de inventario.
3. La integración entre n8n y PostgreSQL permitió mantener la información centralizada y actualizada en tiempo real, garantizando un mejor seguimiento de los productos, alertas generadas y registros de auditoría.
4. La implementación de alertas automáticas para productos con stock bajo y medicamentos próximos a vencer contribuye a una mejor toma de decisiones y ayuda a prevenir pérdidas económicas ocasionadas por desabastecimiento o vencimiento de productos.
5. El sistema de generación de reportes y el manejo automatizado de errores permiten mantener un mayor control sobre el funcionamiento de la aplicación, facilitando el monitoreo, la supervisión y la detección de incidencias dentro del proceso automatizado.

RECOMENDACIONES

1. Implementar un sistema de autenticación y control de usuarios que permita restringir el acceso a las diferentes funciones del sistema según el rol de cada usuario.
2. Incorporar mecanismos de respaldo automático de la base de datos para garantizar la recuperación de la información ante posibles fallos o pérdidas de datos.
3. Integrar servicios de notificación por correo electrónico o aplicaciones de mensajería para informar automáticamente sobre alertas críticas relacionadas con stock bajo, vencimientos próximos o errores del sistema.
4. Desplegar la solución en un entorno de producción en la nube para mejorar la disponibilidad, accesibilidad y escalabilidad del sistema cuando sea utilizado por múltiples usuarios.
5. Continuar ampliando las funcionalidades del proyecto mediante la incorporación de módulos adicionales, como gestión de ventas, proveedores, compras y control de usuarios, con el fin de convertir la solución en un sistema integral para la administración de farmacias.